

# ProcessTensors.jl: An ITensor-native framework for non-Markovian open quantum dynamics

Gauthameshwar S. <sup>1,2,\*</sup>, Aaron Sander <sup>3</sup>, John F. Kam <sup>4,5</sup>, Dario Poletti <sup>1,2,6</sup>, and Kavan Modi <sup>1,2,†</sup>

<sup>1</sup> Singapore University of Technology and Design, Singapore

<sup>2</sup> Centre for Quantum Technologies, Singapore

<sup>3</sup> Technical University of Munich, Munich, Germany

<sup>4</sup> Monash University, Melbourne, Australia

<sup>5</sup> Agency for Science, Technology and Research (A\*STAR), Singapore

<sup>6</sup> MajuLab, CNRS-UNS-NUS-NTU International Joint Research Unit, Singapore

\* [gauthameshwar\\_s@mymail.sutd.edu.sg](mailto:gauthameshwar_s@mymail.sutd.edu.sg), † [kavan\\_modi@sutd.edu.sg](mailto:kavan_modi@sutd.edu.sg)

## Abstract

We present ProcessTensors.jl, a Julia package for learning, constructing, and using process tensors with ITensor. The paper introduces process tensors from the ground up and connects the equations to tensor diagrams, tagged ITensor indices, and working code. Through guided examples, users can build process tensors, apply different sequences of quantum instruments, use testers with ancillary systems, and inspect memory across the temporal bonds. We implement exact and compressed ACE constructions and benchmark them against established software. We also review the current process-tensor software landscape and explain how the package can be extended with further PT-MPO construction methods, including ideas based on TEMPO, TEDOPA, and periodic process tensors. Our goal is to make process tensors approachable for new users while providing a clear foundation for future development of this package.

Copyright attribution to authors.

This work is a submission to SciPost Physics Codebases.

License information to appear upon publication.

Publication information to appear upon publication.

Received Date

Accepted Date

Published Date

## Contents

|          |                                                       |           |
|----------|-------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                                   | <b>2</b>  |
| <b>2</b> | <b>Liouville-space and tensor-network foundations</b> | <b>5</b>  |
| 2.1      | A graphical dictionary for tensors                    | 5         |
| 2.2      | From density operators to Liouville vectors           | 6         |
| 2.3      | Traces and observables as closures                    | 8         |
| 2.4      | Left and right actions                                | 9         |
| 2.5      | Quantum operations as Liouville superoperators        | 10        |
| 2.6      | Dynamics in Liouville space                           | 10        |
| 2.7      | The equation–diagram–code dictionary                  | 12        |
| <b>3</b> | <b>Software architecture</b>                          | <b>12</b> |
| 3.1      | ITensor as the tensor-network substrate               | 12        |

|           |                                                              |           |
|-----------|--------------------------------------------------------------|-----------|
| 3.2       | Hilbert- and Liouville-space objects                         | 12        |
| 3.3       | Systems and environments                                     | 13        |
| 3.4       | A backend-oriented process-tensor API                        | 13        |
| <b>4</b>  | <b>Process tensors as first-class tensor-network objects</b> | <b>13</b> |
| 4.1       | From a channel to a process                                  | 13        |
| 4.2       | Anatomy of ProcessTensor                                     | 13        |
| 4.3       | Building a small process tensor                              | 13        |
| 4.4       | Construction backends: dense reference and ACE               | 13        |
| <b>5</b>  | <b>Quantum instruments</b>                                   | <b>13</b> |
| 5.1       | Operational meaning                                          | 13        |
| 5.2       | Instrument organisation in ProcessTensors.jl                 | 13        |
| 5.3       | InstrumentSeq: a temporal analogue of OpSum                  | 13        |
| <b>6</b>  | <b>Contracting and evaluating processes</b>                  | <b>13</b> |
| 6.1       | General contraction with evaluate_process                    | 13        |
| 6.2       | Reduced dynamics with evolve                                 | 13        |
| 6.3       | Open process legs                                            | 13        |
| <b>7</b>  | <b>Common process-tensor workflows</b>                       | <b>13</b> |
| 7.1       | Final reduced density matrices                               | 13        |
| 7.2       | Observable expectation values                                | 13        |
| 7.3       | Probabilities and trace-non-preserving branches              | 13        |
| 7.4       | Multi-time correlation functions                             | 13        |
| <b>8</b>  | <b>Testers and enlarged interventions</b>                    | <b>13</b> |
| <b>9</b>  | <b>Characterising process tensors and quantum memory</b>     | <b>13</b> |
| <b>10</b> | <b>Validation and ACE benchmarks</b>                         | <b>14</b> |
| 10.1      | Correctness against dense and exact references               | 14        |
| 10.2      | Compression accuracy and temporal bond dimension             | 14        |
| 10.3      | Runtime and memory scaling                                   | 14        |
| 10.4      | Comparison with the C++ ACE toolkit                          | 14        |
| <b>11</b> | <b>Conclusion and outlook</b>                                | <b>14</b> |
| <b>A</b>  | <b>Broader Hilbert- and Liouville-space functionality</b>    | <b>14</b> |
| <b>B</b>  | <b>Vectorisation conventions and index orientation</b>       | <b>14</b> |
|           | <b>References</b>                                            | <b>14</b> |

---

## 1 Introduction

In principle, describing an open quantum system is easy: write down a giant wavefunction for the entire universe, place it in an even more gigantic Hilbert space, and evolve it according to the Schrödinger equation. Somewhere inside that wavefunction lies everything one could

wish to know about the system of interest. In practice, of course, we cannot keep track of every surrounding particle, lattice vibration, stray electromagnetic field, or experimental imperfection and calculate their joint correlated dynamics. We therefore do what physicists have always done when the universe becomes inconveniently large: draw a boundary. The degrees of freedom we wish to study are called the *system*; everything beyond this boundary is called the *environment* or *bath*. Open quantum-system theory asks how the bath affects the system without requiring us to simulate the bath in all its microscopic detail.

The most familiar answers we would see in this regard are those that rely on weak-coupling, short-memory, or related approximations on the baths that lead to time-local master equations. In the Markovian limit, the system does not need a record of its earlier history in order to determine what happens next. Markovian master equations provide an extremely successful workhorse for open quantum dynamics, with the Gorini–Kossakowski–Sudarshan–Lindblad form supplying the standard generator of quantum dynamical semigroups [1, 2]. Nevertheless, weak coupling and a clean separation of timescales are not available in every problem. Structured reservoirs, strong system–bath coupling, slow environmental modes, and delayed feedback can all preserve information about earlier dynamics [3, 4]. In these regimes, memory is part of the physics rather than a correction that can safely be ignored.

The difficulty is not merely that the environment remembers, but that the system may give us no indication of *what* it remembers if we trace out the bath. To see this, consider two joint system–environment states at some time  $t_1$ ,

$$\rho_{SE}^{(1)}(t_1) \quad \text{and} \quad \rho_{SE}^{(2)}(t_1), \quad (1)$$

which reduce to exactly the same system state,

$$\text{Tr}_E[\rho_{SE}^{(1)}(t_1)] = \text{Tr}_E[\rho_{SE}^{(2)}(t_1)] = \rho_S(t_1). \quad (2)$$

As far as the system density matrix is concerned, these two situations are indistinguishable. The environment, however, may disagree. It can retain different states or different correlations with the system in the two cases. Consequently, evolving both joint states with the same system–environment unitary need not produce the same reduced state at a later time:

$$\text{Tr}_E[U_{2:1}\rho_{SE}^{(1)}(t_1)U_{2:1}^\dagger] \neq \text{Tr}_E[U_{2:1}\rho_{SE}^{(2)}(t_1)U_{2:1}^\dagger]. \quad (3)$$

Thus, even perfect knowledge of the present system state need not be sufficient to predict its future. The information distinguishing the two experiments has simply moved somewhere beyond the boundary we drew earlier.

The limitation becomes sharper when we intervene our system along the way. Imagine performing two experiments with different histories and, at an intermediate time  $t_1$ , discarding the system state and preparing the same fresh state  $P$  in both. This operation—often called a *causal break*—removes any information that could pass directly from the past to the future through the system. Immediately afterward, both experiments therefore contain the same reduced state,

$$\rho_S^{(1)}(t_1^+) = \rho_S^{(2)}(t_1^+) = P. \quad (4)$$

Nevertheless, the intervention acts on a system that may already be correlated with its environment. A measurement outcome, for example, can condition the environment into a corresponding state, while repreparing  $P$  makes the system look identical in every run. If the later dynamics still depends on the earlier preparation or measurement outcome, then the memory must have travelled across the causal break through the environment. Such memory can remain hidden in an uninterrupted reduced trajectory, and can even occur when familiar two-time criteria describe the unperturbed evolution as Markovian [5].

This leaves a state-to-state description with an awkward question. What map should take  $P$  at  $t_1$  to the state at  $t_2$ , if the same  $P$  can lead to different futures depending on what happened before it was prepared? There is no contradiction in quantum mechanics here; rather, we have asked the reduced state to remember information that it no longer contains. The missing input is the history of interventions—the preparations, controls, and measurements through which the system and its environment arrived at the present experiment. A complete description of the quantum process must therefore accept an entire ordered sequence of operations, rather than only a density matrix at one particular time.

The process tensor provides precisely such a description [6, 7]. For a sequence of interventions  $(\mathcal{A}_0, \mathcal{A}_1, \dots, \mathcal{A}_{n-1})$ , it returns the final system state,

$$\rho_S(t_n) = \mathcal{T}_{n:0}[\mathcal{A}_{n-1}, \dots, \mathcal{A}_0], \quad (5)$$

or, when the interventions contain measurements, the corresponding outcome probabilities and conditional states. A reduced trajectory tells us what happened under one prescribed history; a process tensor tells us what could have happened had we intervened differently along the way. It does not reveal every microscopic bath degree of freedom, but retains their complete influence on experiments performed through the system (the mathematical construction behind this statement—including its Choi representation and temporal input and output spaces—will be developed carefully in Sec. 4. For the present discussion, its operational meaning is enough). This makes causal breaks, multi-time correlations, instrument sequences, Markov order, and correlations across temporal partitions accessible within one description. In the Markovian limit, the process separates into independent transformations between successive times; otherwise, its temporal parts remain correlated, recording how information stored in the environment can return later. Memory thus becomes part of the object being manipulated rather than a feature inferred from a single trajectory.

The generality of a process tensor comes at an exponential cost: every additional intervention time introduces another pair of system indices, thus causing the exponential growth of the representation space of a process tensor, similar to how a many-body wavefunction space grows with the number of particles. Tensor networks make this manageable by representing the process as a matrix product operator in time, or PT-MPO [8]. The open legs provide the slots for system operations, while the internal bonds carry correlations between different parts of the process. When these temporal correlations are compressible, singular-value truncations reduce the representation to a controllable bond dimension. Different algorithms construct this PT-MPO by exploiting different structures of the environment: PT-TEMPO contracts the influence functional of Gaussian bosonic baths [8, 9], ACE combines and compresses the influence of independent bath modes [10], and time-translation-invariant approaches target long-time processes using repeating tensors [11]. The process tensor is therefore not made small by assumption; it is made useful whenever the environment stores its relevant memory in a compressible temporal space.

The principal open-source packages that construct reusable PT-MPOs are OQuPy, which implements PT-TEMPO for Gaussian bosonic environments [12], and the C++ ACE toolkit, which supports ACE, PT-TEMPO, and related contraction schemes for a broader class of independent bath modes [13]. Packages such as `QuantumDynamics.jl` and `MPSDynamics.jl` provide complementary influence-functional and explicit-environment tensor-network methods [14, 15]. Broader comparisons of these algorithms, software packages, and their detailed applicable regimes are already available in recent reviews [16, 17].

Then enters the hero of this paper, `ProcessTensors.jl`. BA BA BAA BAAMMMMMMM. We are so easy to use. We also have a lot of documentation and examples. Someone just after learning basic Julia and reading this paper, can already install and use our software to fiddle with process tensors. We are the best. Just Kidding. The remaining part of the intro will be completed once we finalise the main contents of this scipost paper. Stay tuned!

## 2 Liouville-space and tensor-network foundations

The process tensor is a map acting on other maps. Before constructing such an object, it is useful to place density matrices and quantum operations on equal footing. Moving to the Liouville space does precisely this: density matrices become state-like vectors, while operations on density matrices become ordinary linear operators. This change of language is especially natural for tensor networks, because the same ideas can be read in three equivalent ways—as an equation, as a tensor diagram, and as the corresponding ITensor code. We develop these three descriptions together below. Our graphical conventions are adapted from the open-system tensor calculus of Ref. [18], with time and contractions drawn from right to left to match the convention used throughout this paper. In case the readers are not familiar with ITensors and their notations of sites and indices, they are encouraged to read the documentation of the ITensor library [cite ITensor docs, and also the original papers]. Alternatively, they can also go through the tutorials in the stable docs of our package [add the link to the stable docs].

### 2.1 A graphical dictionary for tensors

A tensor is represented by a shape with one open wire for every index. In our conventions, the direction of a wire records which dual space that leg belongs to: a leg pointing to the left is a ket index [Fig. 1a], whereas a leg pointing to the right is a bra index [Fig. 1b]. Joining two wires means summing over their shared index. For example, ordinary matrix multiplication,

$$(AB)_{ij} = A_{ik}B_{kj}, \quad (6)$$

is drawn by joining the right-hand wire of  $A$  to the left-hand wire of  $B$  [Fig. 1e]. Stacking two disconnected tensors vertically, with ket wires on the left and bra wires on the right, represents their tensor product [Fig. 1f]. Closing the two wires of an operator on the same index with a loop gives its trace [Fig. 1g],

$$\text{Tr}(A) = A_{ii}. \quad (7)$$

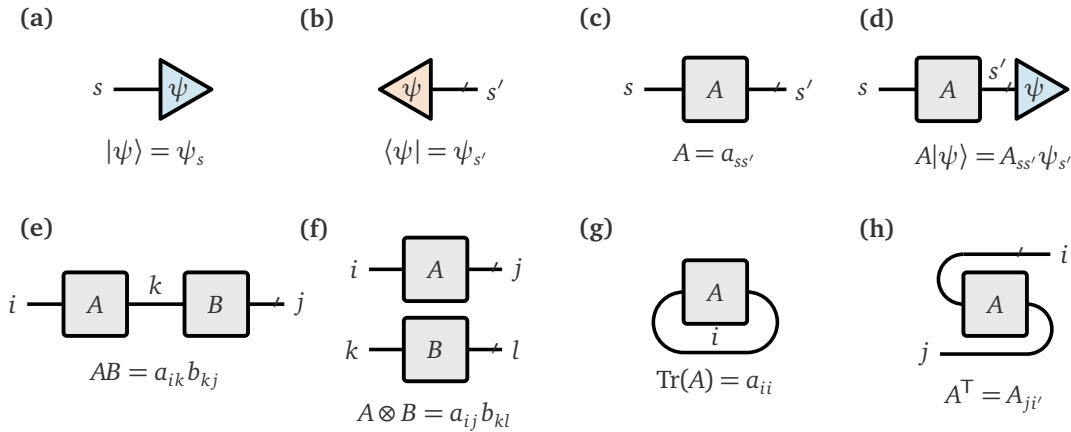


Figure 1: Graphical dictionary used throughout this work. (a) A ket  $|\psi\rangle$  with unprimed index  $s$ , (b) A bra  $\langle\psi|$  with primed index  $s'$ , (c) An operator  $A$  with a ket wire  $s$  on the left and a bra wire  $s'$  on the right. (d) Contracting the primed wire of  $A$  with  $|\psi\rangle$ . (e) The matrix multiplication  $AB = a_{ik}b_{kj}$ . (f) The tensor product  $A \otimes B = a_{ij}b_{kl}$ , drawn by stacking  $A$  above  $B$ . (g) The trace  $\text{Tr}(A) = a_{ii}$ . (h) The transpose  $A^T = A_{ji'}$ , obtained by bending the left and right wires to the opposite directions.

Equations (6) and (7), Fig. 1, and Listing 1 express the same basic rule: tensors contract when their indices match. Wire bending deserves one warning though, when we implement

them in ITensor codes. A bend represents a basis-dependent transpose (swapping two indices), complex conjugation changes the tensor entries, and combining the two gives the Hermitian adjoint,  $A^\dagger = (A^*)^T$ . An ITensor prime operation, by contrast, only changes the identity of an index. It does not transpose or conjugate the tensor. For achieving a dagger, we need to use the Hermitian adjoint function dag and then prime the indices. This distinction becomes important when we use an unprimed index for the input and a primed copy for its output when constructing operator ITensors.

Julia

```

1  using ITensors
2
3  s = Index(2, "s")
4
5  psi = ITensor([1.0, 0.0], s) # (a) ket |psi> = psi_s
6
7  psidag = dag(prime(psi)) # (b) bra <psi| = psi_s'
8
9  A = random_itensor(s, prime(s)) # (c) operator A = a_ss'
10
11 Apsi = A * prime(psi) # (d) A|psi> = A_ss' psi_s'
12
13 i = Index(2, "i")
14 k = Index(2, "k")
15 j = Index(2, "j")
16 AB = random_itensor(i, k) * random_itensor(k, j) # (e) AB = a_ik b_kj
17
18 l = Index(2, "l")
19 AxB = random_itensor(i, j) * random_itensor(k, l) # (f) A ox B = a_ij
    ↪ b_kl (no shared index)
20
21 TrA = A * delta(s, prime(s)) # (g) Tr(A) = a_ii
22
23 AT = conj(dag(A)) # (h) AT = a_ji

```

Listing 1: ITensor codes to realise tensor operations in Fig. 1(a)–(h).

## 2.2 From density operators to Liouville vectors

Let  $\mathcal{H}$  be a  $d$ -dimensional Hilbert space. A density operator is an element of the operator space  $\mathcal{B}(\mathcal{H})$ . This operator space is itself a Hilbert space when equipped with the Hilbert–Schmidt inner product

$$\langle\langle A|B \rangle\rangle = \text{Tr}(A^\dagger B). \quad (8)$$

Because  $\dim \mathcal{B}(\mathcal{H}) = d^2$ , an operator can be treated as a vector in a larger space. We denote this vectorisation by  $A \mapsto |A\rangle\rangle$ .

Different vectorisation conventions arrange the matrix entries differently. `ProcessTensors.jl` follows Julia’s column-major ordering. In a fixed basis  $\{|j\rangle\}$ ,

$$|A\rangle\rangle \equiv \text{vec}(A) = \sum_{j,k} A_{jk} |k\rangle \otimes |j\rangle, \quad (9)$$

**Tensor-diagram placeholder: vectorisation by wire bending**

Future source: `figures/liouville_vectorisation.tikz`.

Panel (a): draw  $A$  as an operator box with an input and an output. Panel (b): attach a cup representing  $|I\rangle\rangle$  to the input wire and bend it into a second output, producing  $|A\rangle\rangle$ . Panel (c): set  $A = I$  and show that the remaining curved wire is the vectorised identity. Label the two output components in the order  $|k\rangle \otimes |j\rangle$ , matching the column-major convention of Eq. (9).

Figure 2: Column-major vectorisation as a wire-bending operation. The bend converts an operator into a vector in  $\mathcal{H} \otimes \mathcal{H}^*$ .

**Tensor-diagram placeholder: density MPO to Liouville MPS**

Future source: `figures/mpo_to_liouville_mps.tikz`.

Draw a three-site density MPO. At each site show an unprimed Hilbert index  $s_j$ , a primed index  $s'_j$ , and the horizontal MPO bond. Attach a three-legged combiner  $C_j$  to  $s_j, s'_j$ , producing one thick Liouville leg  $\ell_j$ . The final panel is an `MPS{Liouville}` with the original horizontal bonds and one Liouville leg per site. Annotate one example with `Site,S=1/2,n=1` and `Liouv,ptype=S=1/2,n=1`; also mark  $\dim s_j = d_j$  and  $\dim \ell_j = d_j^2$ .

Figure 3: Local vectorisation used by `ProcessTensors.jl`. A combiner fuses the ket and bra indices of each density-MPO tensor into one Liouville index.

so that

$$\begin{pmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{pmatrix} \mapsto \begin{pmatrix} A_{11} \\ A_{21} \\ A_{12} \\ A_{22} \end{pmatrix}. \quad (10)$$

Equivalently,  $\text{vec}(|j\rangle\langle k|) = |k\rangle \otimes |j\rangle$ . Stating this ordering explicitly fixes every transpose that appears later.

Graphically, vectorisation bends the input wire of an operator until both wires point in the same direction. The bend is the unnormalised maximally entangled state

$$|I\rangle\rangle = \sum_j |j\rangle \otimes |j\rangle, \quad (11)$$

which is the vectorised identity operator. Figure 2 pairs this graphical operation with Eqs. (9) and (11).

For a many-site tensor network, `ProcessTensors.jl` performs this reorganisation locally. A Hilbert-space density matrix is stored as an `MPO{Hilbert}` with a ket and a primed bra index on every site. A combiner fuses each local pair into a single Liouville index of dimension  $d_j^2$ , producing an `MPS{Liouville}`:

$$C_j : \mathcal{H}_j \otimes \mathcal{H}_j^* \longrightarrow \mathcal{L}_j, \quad \dim \mathcal{L}_j = d_j^2. \quad (12)$$

This local fusion is shown in Fig. 3.



**Tensor-diagram placeholder: closing a Liouville leg**

Future source: figures/liouville\_closures.tikz.

Draw the same  $|\rho\rangle\rangle$  tensor in three panels. In (a), close its Liouville leg with  $\langle\langle I|$  to obtain the scalar  $\text{Tr} \rho$ . In (b), close it with  $\langle\langle O|$  to obtain  $\text{Tr}(O\rho)$ . In (c), leave the leg open and label the result  $|\rho\rangle\rangle$ . Use the same visual shapes later for `trace_out`, `observable_measurement`, and `open_output`.

Figure 4: The trace, an expectation value, and an open Liouville output are three different closures of the same tensor-network leg.

```

1 sites = siteinds("S=1/2", 1)
2 psi = MPS(ComplexF64[0.6, 0.8im], sites)
3
4 rho = to_dm(psi)           # MPO{Hilbert}
5 sites_L = liouv_sites(sites) # local dimension d^2
6 rho_L = to_liouville(rho; sites=sites_L)
7                                     # MPS{Liouville}
8
9 rho_back = to_hilbert(rho_L) # undo the local fusion

```

Julia

Listing 2: Hilbert-to-Liouville conversion in `ProcessTensors.jl`.

The Liouville indices should be created once and then reused throughout a calculation. Two independent calls to `liouv_sites` produce indices with the same dimensions and tags but different identities, which is enough to prevent an intended `ITensor` contraction.

### 2.3 Traces and observables as closures

Once a density operator has become a Liouville vector, traces and expectation values become inner products. From Eq. (8),

$$\text{Tr}(\rho) = \langle\langle I|\rho\rangle\rangle, \quad \langle\langle O\rangle = \text{Tr}(O\rho) = \langle\langle O|\rho\rangle\rangle, \quad (13)$$

where the last equality assumes the observable  $O = O^\dagger$ . Graphically, the vectorised identity closes a Liouville leg and returns the trace; replacing it by a vectorised observable returns its expectation value. Leaving the same leg open returns a state-like Liouville object instead. These three choices are shown together in Fig. 4 because they later become three basic ways of interrogating a process tensor.

```

1 Id = OpSum()
2 Id += 1.0, "Id", 1
3
4 O = OpSum()
5 O += 1.0, "Sz", 1
6
7 Id_L = to_liouville(MPO(Id, sites); sites=sites_L)
8 O_L = to_liouville(MPO(O, sites); sites=sites_L)
9

```

Julia



**Tensor-diagram placeholder: wire sliding and Roth's lemma**

Future source: `figures/left_right_wire_sliding.tikz`.

First row: place  $A$  on one arm of the cup representing  $|I\rangle\rangle$ , slide it continuously around the bend, and show  $A^T$  on the other arm. Second row: draw the ordinary product  $A\rho B$ , bend the input of  $\rho$ , and obtain a doubled Liouville wire with  $A$  on one component and  $B^T$  on the other. Final two panels should isolate  $A_L$  and  $B_R$  and match Eqs. (16)–(17).

Figure 5: Wire bending explains the transpose in right multiplication and gives a graphical proof of Eq. (15).

```
10 normalisation = inner(Id_L, rho_L)
11 expectation   = inner(O_L, rho_L)
```

Listing 3: Traces and observables as Liouville inner products.

The operations in Fig. 4 will therefore require no new graphical language later. A trace, measurement insertion, or open output will simply be attached to a chosen temporal leg of the process tensor.

## 2.4 Left and right actions

The transpose introduced by vectorisation can be understood directly by sliding an operator around a bend. For the vectorised identity,

$$(I \otimes A)|I\rangle\rangle = (A^T \otimes I)|I\rangle\rangle. \quad (14)$$

An operator moved from one side of the cup to the other therefore emerges transposed. Applying this rule to a product of three operators gives the column-vectorisation identity

$$\text{vec}(A\rho B) = (B^T \otimes A)|\rho\rangle\rangle. \quad (15)$$

The two special cases used repeatedly in this work are

$$A\rho \longleftrightarrow (I \otimes A)|\rho\rangle\rangle \equiv A_L|\rho\rangle\rangle, \quad (16)$$

$$\rho B \longleftrightarrow (B^T \otimes I)|\rho\rangle\rangle \equiv B_R|\rho\rangle\rangle. \quad (17)$$

Figure 5 provides the graphical derivation rather than asking the reader to memorise where the transpose belongs.

```
1 sL = sites_L[1]
2
3 A_L = op("Sx_L", sL) # vec(Sx * rho)
4 B_R = op("Sz_R", sL) # vec(rho * Sz)
```

Julia

Listing 4: Package notation for left and right multiplication.

The suffixes `_L` and `_R` refer to multiplication from the left and right of the density matrix *before* vectorisation. They do not refer to the geometric left and right sides of an ITensor.

**Tensor-diagram placeholder: a quantum operation in Liouville space**

Future source: figures/kraus\_liouville\_map.tikz.

Panel (a): draw  $K_\mu \rho K_\mu^\dagger$  as a density-matrix network with the Kraus label  $\mu$  visibly summed. Panel (b): bend the density-matrix wire and show the superoperator  $K_\mu^* \otimes K_\mu$  acting on  $|\rho\rangle\rangle$ . Panel (c): attach the trace cap  $\langle\langle I|$  after a trace-preserving map and simplify it to the same cap before the map, illustrating Eq. (20).

Figure 6: A completely positive map becomes an ordinary operator on Liouville space. Trace preservation is expressed by the invariance of the identity effect.

## 2.5 Quantum operations as Liouville superoperators

A quantum operation is a completely positive map that may decrease the trace of a state. It can therefore describe either deterministic evolution or a particular outcome of a measurement. Any such map admits an operator-sum representation

$$\mathcal{A}(\rho) = \sum_{\mu} K_{\mu} \rho K_{\mu}^{\dagger}. \quad (18)$$

Using Eq. (15), its Liouville-space matrix is

$$A = \sum_{\mu} K_{\mu}^* \otimes K_{\mu}, \quad |\mathcal{A}(\rho)\rangle\rangle = A|\rho\rangle\rangle. \quad (19)$$

The doubled tensor network in Fig. 6 makes the two copies of each Kraus operator and the sum over  $\mu$  explicit. If  $\sum_{\mu} K_{\mu}^{\dagger} K_{\mu} = I$ , the operation is trace preserving and is a quantum channel. In Liouville notation this condition is

$$\langle\langle I|A = \langle\langle I|. \quad (20)$$

For a single physical operator  $K$ , the package constructs the doubled term  $K^* \otimes K$  through the `_Jump` suffix:

```
1 K_jump = op("S-Jump", sites_L[1]) # conj(S-) kron S-
```

Julia

Listing 5: A local doubled operator in Liouville space.

General preparations, controls, and measurement outcomes will be assembled from the same ingredients when instruments are introduced in Sec. 4.

## 2.6 Dynamics in Liouville space

For a closed system, the density matrix obeys the von Neumann equation

$$\frac{d\rho}{dt} = -i[H, \rho]. \quad (21)$$

Equations (16) and (17) turn the commutator into a linear generator on Liouville space,

$$\frac{d}{dt}|\rho(t)\rangle\rangle = \mathcal{L}_H|\rho(t)\rangle\rangle, \quad \mathcal{L}_H = -iH_L + iH_R. \quad (22)$$

The two terms in this generator are shown separately in Fig. 7; their signs are inherited directly from the commutator.

**Tensor-diagram placeholder: construction of a Liouvillian**

Future source: `figures/liouvillian_terms.tikz`.

Top row: draw the Hamiltonian commutator as the sum of two doubled-wire diagrams,  $-iH_L$  and  $+iH_R$ . Bottom row: draw the three pieces of one GKSL term—the jump  $L\rho L^\dagger$ , the left anticommutator contribution, and the right anticommutator contribution—with their coefficients. Use the same left/right conventions as Fig. 5.

Figure 7: Hamiltonian and dissipative contributions to a Liouville-space generator. Every term is an operator acting on the doubled physical wire.

Markovian dissipation can be included through the Gorini–Kossakowski–Sudarshan–Lindblad generator,

$$\mathcal{L}(\rho) = -i[H, \rho] + \sum_{\mu} \gamma_{\mu} \left( L_{\mu} \rho L_{\mu}^{\dagger} - \frac{1}{2} L_{\mu}^{\dagger} L_{\mu} \rho - \frac{1}{2} \rho L_{\mu}^{\dagger} L_{\mu} \right). \quad (23)$$

Here its role is not to reintroduce the Markov approximation, but to establish the common Liouville-space representation used by the package: Hamiltonian terms, dissipators, and later the full system–bath propagator are all linear operators acting on vectorised density matrices.

Julia

```

1  H = OpSum()
2  H += 0.7, "Sx", 1
3  H += 0.2, "Sz", 1
4
5  L_H = liouvillian_mpo(H, sites_L)
6  L_open = liouvillian_mpo(
7      H,
8      sites_L;
9      jump_ops=[(0.1, "S-", 1)],
10 )

```

Listing 6: Constructing Hamiltonian and dissipative Liouvillians.

For a short time step  $\Delta t$ , the propagator is  $U_k = \exp(\Delta t \mathcal{L}_k)$ . A prescribed trajectory is therefore the contraction

$$|\rho(t_n)\rangle\rangle = U_{n-1} \cdots U_1 U_0 |\rho(t_0)\rangle\rangle. \quad (24)$$

Figure 8 draws this equation as a temporal tensor network.

Julia

```

1  dt = 0.1
2  U = liouvillian_propagator(
3      H,
4      sites_L,
5      dt;
6      jump_ops=[(0.1, "S-", 1)],
7  )
8
9  rho_next = apply(U, rho_L)

```

Listing 7: One discrete Liouville-space propagation step.

**Tensor-diagram placeholder: a contracted Liouville trajectory**

Future source: figures/liouville\_trajectory.tikz.

Draw  $|\rho(t_0)\rangle\rangle$  followed from left to right by a chain of two-legged propagator tensors  $U_0, U_1, \dots, U_{n-1}$ , ending in  $|\rho(t_n)\rangle\rangle$ . Label the connecting Liouville indices and the discrete times  $t_0, \dots, t_n$ . Beneath it, draw a faded preview in which the system operations between time steps have been removed and replaced by open input/output legs. This preview will become the process tensor in the next section.

Figure 8: A reduced trajectory as a fully contracted temporal network. The process-tensor construction will retain the environmental part of this network while leaving the intermediate system operations open.

Table 1: Hilbert-space expressions, Liouville-space representations, and their `ProcessTensors.jl` counterparts.

| Hilbert-space object | Liouville-space object                | Package representation          |
|----------------------|---------------------------------------|---------------------------------|
| $ \psi\rangle$       | state vector                          | <code>MPS{Hilbert}</code>       |
| $\rho$               | $ \rho\rangle\rangle$                 | <code>MPS{Liouville}</code>     |
| density operator     | ket–bra legs fused locally            | <code>to_liouville</code>       |
| $A\rho$              | $A_L \rho\rangle\rangle$              | <code>"A_L"</code>              |
| $\rho A$             | $A_R \rho\rangle\rangle$              | <code>"A_R"</code>              |
| $A\rho A^\dagger$    | $(A^* \otimes A) \rho\rangle\rangle$  | <code>"A_Jump"</code>           |
| $\text{Tr } \rho$    | $\langle\langle I \rho\rangle\rangle$ | <code>inner(Id_L, rho_L)</code> |
| $-i[H, \rho]$        | $\mathcal{L}_H \rho\rangle\rangle$    | <code>liouvillian_mpo</code>    |

## 2.7 The equation–diagram–code dictionary

Table 1 collects the conventions established in this section. It is also a compact reference for the process-tensor diagrams that follow.

Liouville space has now turned density operators into vectors, physical operations into tensors acting on those vectors, and time evolution into a network of contractions. In Eq. (24), every operation between two times has already been chosen. We now leave those choices open. The resulting multi-time object is the process tensor.

## 3 Software architecture

There is no strict length limitation, but the authors are strongly encouraged to keep contents to the strict minimum necessary for peers to reproduce the research described in the paper.

### 3.1 ITensor as the tensor-network substrate

You are free to use dividers as you see fit.

### 3.2 Hilbert- and Liouville-space objects

Figures should only occupy the strictly necessary space, in any case individually fitting on a single page. Each figure item should be appropriately labeled and accompanied by a descriptive caption. **SciPost does not accept creative or promotional figures or artist’s impressions;**

on the other hand, technical drawings and scientifically accurate representations are encouraged.

### 3.3 Systems and environments

### 3.4 A backend-oriented process-tensor API

## 4 Process tensors as first-class tensor-network objects

### 4.1 From a channel to a process

### 4.2 Anatomy of `ProcessTensor`

### 4.3 Building a small process tensor

### 4.4 Construction backends: dense reference and ACE

## 5 Quantum instruments

### 5.1 Operational meaning

### 5.2 Instrument organisation in `ProcessTensors.jl`

### 5.3 `InstrumentSeq`: a temporal analogue of `OpSum`

## 6 Contracting and evaluating processes

### 6.1 General contraction with `evaluate_process`

### 6.2 Reduced dynamics with `evolve`

### 6.3 Open process legs

## 7 Common process-tensor workflows

### 7.1 Final reduced density matrices

### 7.2 Observable expectation values

### 7.3 Probabilities and trace-non-preserving branches

### 7.4 Multi-time correlation functions

## 8 Testers and enlarged interventions

Status: planned for the paper release; API under development.

## 9 Characterising process tensors and quantum memory

Section lead: Aaron Sander. The text below is a structural placeholder and should not be read as claiming currently released package functionality.

## 10 Validation and ACE benchmarks

10.1 Correctness against dense and exact references

10.2 Compression accuracy and temporal bond dimension

10.3 Runtime and memory scaling

10.4 Comparison with the C++ ACE toolkit

## 11 Conclusion and outlook

You must include a conclusion.

## Acknowledgements

Acknowledgements should follow immediately after the conclusion.

**Author contributions** This is optional. If desired, contributions should be succinctly described in a single short paragraph, using author initials.

**Funding information** Authors are required to provide funding information, including relevant agencies and grant numbers with linked author's initials. Correctly-provided data will be linked to funders listed in the [Fundref registry](#).

**Code and data availability** The development repository is available at <https://github.com/Gauthameshwar/ProcessTensors.jl> and the documentation at <https://gauthameshwar.github.io/ProcessTensors.jl/stable/>. The final paper will cite an immutable release and archive the scripts/data used for all benchmark figures. [TODO: insert release DOI/Zenodo or SciPost codebase DOI at submission.]

## A Broader Hilbert- and Liouville-space functionality

Add material which is better left outside the main text in a series of Appendices labeled by capital letters.

## B Vectorisation conventions and index orientation

## References

- [1] V. Gorini, A. Kossakowski and E. C. G. Sudarshan, *Completely positive dynamical semigroups of N-level systems*, Journal of Mathematical Physics **17**(5), 821 (1976), doi:[10.1063/1.522979](#).
- [2] G. Lindblad, *On the generators of quantum dynamical semigroups*, Communications in Mathematical Physics **48**(2), 119 (1976), doi:[10.1007/BF01608499](#).

- [3] Á. Rivas, S. F. Huelga and M. B. Plenio, *Quantum non-markovianity: Characterization, quantification and detection*, Reports on Progress in Physics **77**(9), 094001 (2014), doi:[10.1088/0034-4885/77/9/094001](https://doi.org/10.1088/0034-4885/77/9/094001).
- [4] I. de Vega and D. Alonso, *Dynamics of non-markovian open quantum systems*, Reviews of Modern Physics **89**(1), 015001 (2017), doi:[10.1103/RevModPhys.89.015001](https://doi.org/10.1103/RevModPhys.89.015001).
- [5] F. A. Pollock, C. Rodríguez-Rosario, T. Frauenheim, M. Paternostro and K. Modi, *Operational markov condition for quantum processes*, Physical Review Letters **120**(4), 040405 (2018), doi:[10.1103/PhysRevLett.120.040405](https://doi.org/10.1103/PhysRevLett.120.040405).
- [6] F. A. Pollock, C. Rodríguez-Rosario, T. Frauenheim, M. Paternostro and K. Modi, *Non-markovian quantum processes: Complete framework and efficient characterization*, Physical Review A **97**(1), 012127 (2018), doi:[10.1103/PhysRevA.97.012127](https://doi.org/10.1103/PhysRevA.97.012127).
- [7] S. Milz and K. Modi, *Quantum stochastic processes and quantum non-markovian phenomena*, PRX Quantum **2**(3), 030201 (2021), doi:[10.1103/PRXQuantum.2.030201](https://doi.org/10.1103/PRXQuantum.2.030201).
- [8] M. R. Jørgensen and F. A. Pollock, *Exploiting the causal tensor network structure of quantum processes to efficiently simulate non-markovian path integrals*, Physical Review Letters **123**(24), 240602 (2019), doi:[10.1103/PhysRevLett.123.240602](https://doi.org/10.1103/PhysRevLett.123.240602).
- [9] A. Strathearn, P. Kirton, D. Kilda, J. Keeling and B. W. Lovett, *Efficient non-markovian quantum dynamics using time-evolving matrix product operators*, Nature Communications **9**, 3322 (2018), doi:[10.1038/s41467-018-05617-3](https://doi.org/10.1038/s41467-018-05617-3).
- [10] M. Cygorek, M. Cosacchi, A. Vagov, V. M. Axt, B. W. Lovett, J. Keeling and E. M. Gauger, *Simulation of open quantum systems by automated compression of arbitrary environments*, Nature Physics **18**, 662 (2022), doi:[10.1038/s41567-022-01544-9](https://doi.org/10.1038/s41567-022-01544-9).
- [11] V. Link, H.-H. Tu and W. T. Strunz, *Open quantum system dynamics from infinite tensor network contraction*, Physical Review Letters **132**(20), 200403 (2024), doi:[10.1103/PhysRevLett.132.200403](https://doi.org/10.1103/PhysRevLett.132.200403), [2307.01802](https://arxiv.org/abs/2307.01802).
- [12] G. E. Fux, P. Fowler-Wright, J. Beckles, E. P. Butler, P. R. Eastham, D. Gribben, J. Keeling, D. Kilda, P. Kirton, E. D. C. Lawrence, B. W. Lovett, E. O'Neill *et al.*, *OQuPy: A Python package to efficiently simulate non-markovian open quantum systems with process tensors*, The Journal of Chemical Physics **161**(12), 124108 (2024), doi:[10.1063/5.0225367](https://doi.org/10.1063/5.0225367).
- [13] M. Cygorek and E. M. Gauger, *ACE: A general-purpose non-markovian open quantum systems simulation toolkit based on process tensors*, The Journal of Chemical Physics **161**(7), 074111 (2024), doi:[10.1063/5.0221182](https://doi.org/10.1063/5.0221182).
- [14] A. Bose, *QuantumDynamics.jl: A modular approach to simulations of dynamics of open quantum systems*, The Journal of Chemical Physics **158**(20), 204113 (2023), doi:[10.1063/5.0151483](https://doi.org/10.1063/5.0151483).
- [15] T. Lacroix, B. Le Dé, A. Riva, A. J. Dunnett and A. W. Chin, *MPSDynamics.jl: Tensor network simulations for finite-temperature (non-markovian) open quantum system dynamics*, The Journal of Chemical Physics **161**(8), 084116 (2024), doi:[10.1063/5.0223107](https://doi.org/10.1063/5.0223107).
- [16] J. Keeling, E. M. Stoudenmire, M.-C. Bañuls and D. R. Reichman, *Process tensor approaches to non-markovian quantum dynamics*, Physical Review X **16**, 020502 (2026), doi:[10.1103/1ncg-11hz](https://doi.org/10.1103/1ncg-11hz).



- [17] T. Lacroix, A. Burgess, N. Lorenzoni, J. Wiercinski, K. Damezin, J. Lim, D. Tamascelli, A. W. Chin, M. Cygorek, B. W. Lovett, J. Keeling, S. F. Huelga *et al.*, *Tensor network methods for non-perturbative dynamics of open quantum systems*, doi:[10.48550/arXiv.2608.09850](https://doi.org/10.48550/arXiv.2608.09850) (2026), [2608.09850](https://arxiv.org/abs/2608.09850).
- [18] C. J. Wood, J. D. Biamonte and D. G. Cory, *Tensor networks and graphical calculus for open quantum systems*, *Quantum Information and Computation* **15**(9-10), 759 (2015), doi:[10.26421/QIC15.9-10-3](https://doi.org/10.26421/QIC15.9-10-3), [1111.6950](https://arxiv.org/abs/1111.6950).